

CSC 1024

PROGRAMMING PRINCIPLES

TITLE OF REPORT:

A PYTHON-BASED INVENTORY MANAGEMENT SYSTEM

Prepared by:

Sian Yi Xuan (23099609)

Tan Wei Ting (23094709)

Tan Wen Xi (23093495)

Teoh En Yi (23051105)

Tiffany Fam Kar Ying (23052301)

Prepared for:

ASSOC. PROF. DR ANWAR P.P. ABDUL MAJEED

Date of Submission:

29 December 2024

1. Add a New Product Function

```

import os
import datetime

# File paths for data storage
PRODUCTS_FILE = "products.txt"
SUPPLIERS_FILE = "suppliers.txt"
ORDERS_FILE = "orders.txt"

# 1.1 Add a New Product Function
def add_product():
    # Prompt for Product ID and validate the format
    product_id = input("Enter Product ID (e.g., P001): ").strip()
    if not product_id.startswith("P") or len(product_id) != 4 or not product_id[1:].isdigit():
        print("Error: To make the Product ID dependent, Product ID must start with 'P'
followed by 3 digits (e.g., P001), please try again.")
        return

    # Prompt for Product Name and validate
    name = input("Enter Product Name: ").strip()
    if not name:
        print("Error: Product name cannot be empty, please try again.")
        return

    # Prompt for Product Description and validate
    description = input("Enter Product Description: ").strip()
    if not description:
        print("Error: Product description cannot be empty, please try again.")
        return

    # Prompt for Stock (Quantity) and validate as a positive integer
    try:
        quantity = int(input("Enter Product Quantity: "))
        if quantity < 1:
            print("Error: Quantity has to be a positive integer, please try again.")
            return
    except ValueError:

        print("Error: Quantity has to be a valid positive integer.")
        return

    # Prompt for Price and validate as a positive number
    try:
        price = float(input("Enter Product Price: "))
        if price < 0:
            print("Error: Price cannot be a negative number, please try again.")
            return
    except ValueError:
        print("Error: Price has to be a valid number.")
        return

```

```

# Check for duplicate Product ID in the existing products file
try:
    with open(PRODUCTS_FILE, "r") as file:
        if product_id in file.read():
            print("Error: Product ID already exists. Please enter a special Product ID, try
again.")
        return
except FileNotFoundError:
    print("Note: products.txt file not found. Creating a new file.")

# Append the new product details to the file
with open(PRODUCTS_FILE, "a") as file:
    file.write(f'{product_id},{name},{description},{quantity},{price}\n')
    print(f'Product added successfully: {product_id}, Product name: {name},
Description: {description}, Quantity: {quantity}, and Price: RM {price:.2f}\n')

```

1. Update Product Details

```

def update_product_details():
    """
    This function updates the details of an existing product in the inventory.
    It reads from 'products.txt', allows the user to modify product details, and writes back to
    the file.
    """
    try:
        # Load existing product data
        with open(PRODUCTS_FILE, 'r') as file:
            products = file.readlines()

        if not products:
            print("No products found in the inventory.")
            return

        # Display existing products
        print("Current Products:")
        product_dict = {}
        for line in products:
            product_data = line.strip().split(',')
            product_dict[product_data[0]] = product_data[1:]
            print(f'ID: {product_data[0]}, Name: {product_data[1]}, Description:
{product_data[2]}, Quantity: {product_data[3]}, Price: {product_data[4]}')

        # Prompt for Product ID to update
        product_id = input("Enter the Product ID to update: ").strip()
        if product_id not in product_dict:
            print("Error: Product ID not found in the inventory.")
            return
    
```

```

# Get current details of the product
current_details = product_dict[product_id]

# Update product details
print("Leave fields blank to keep current values.")
new_name = input(f"Enter new name (current: {current_details[0]}): ") or
current_details[0]
new_description = input(f"Enter new description (current: {current_details[1]}): ") or
current_details[1]
try:
    new_quantity = input(f"Enter new quantity (current: {current_details[2]}): ") or
current_details[2]
    if int(new_quantity) < 0:
        print("Error: Quantity cannot be negative.")
        return
except ValueError:
    print("Error: Quantity must be a valid integer.")
    return
try:
    new_price = input(f"Enter new price (current: {current_details[3]}): ") or
current_details[3]
    if float(new_price) < 0:
        print("Error: Price cannot be negative.")
        return
except ValueError:
    print("Error: Price must be a valid number.")
    return

# Update the product dictionary
product_dict[product_id] = [new_name, new_description, new_quantity, new_price]

# Write updated data back to the file
with open(PRODUCTS_FILE, 'w') as file:
    for pid, details in product_dict.items():
        file.write(f'{pid},{','.join(details)}\n')

print("Product details updated successfully.")

except FileNotFoundError:
    print("Error: 'products.txt' file not found.")
except Exception as e:
    print(f"An unexpected error occurred: {e}")

# Example of how to call the function
# update_product_details()

```

2. Supplier Functions

```
def get_suppliers():
```

```

suppliers = []
with open(SUPPLIERS_FILE, 'r') as f:
    for line in f.readlines()[1:]: # Skip the header
        parts = line.strip().split()
        if len(parts) >= 3:
            supplier_id = parts[0]
            name = parts[1]
            contact = parts[2]
            suppliers.append({"id": supplier_id, "name": name, "contact": contact})
return suppliers

def view_supplier():
    with open(SUPPLIERS_FILE, 'r') as f:
        contents = f.read()
    print("-----")
    print("This is the current supplier details")
    print(contents)
    print("-----")

def is_duplicate(suppliers, supplier_id, name, contact):
    for supplier in suppliers:
        if supplier["id"] == supplier_id:
            print(f"Error: Supplier ID {supplier_id} already exists!")
            return True
        if supplier["name"] == name:
            print(f"Error: Supplier name {name} already exists!")
            return True
        if supplier["contact"] == contact:
            print(f"Error: Supplier contact {contact} already exists!")
            return True
    return False

def add_supplier(supplier_id, name, contact):
    suppliers = get_suppliers()
    if is_duplicate(suppliers, supplier_id, name, contact):
        print("Duplicate entry detected. Please try again with unique details.")
        return
    with open(SUPPLIERS_FILE, 'a') as f:
        formatted_entry = f"{supplier_id:<15} {name:<10} {contact:<12}\n"
        f.write(formatted_entry)
    print("Supplier added successfully!")

def read_supplier():
    with open(SUPPLIERS_FILE, 'r') as f:
        contents = f.read()
    print("\n-----")
    print("Your new supplier information:")
    print(contents)
    print("-----")

```

```

def add_supplier_interactive():
    while True:
        supplier_id = input("Please enter Supplier ID: ").upper()
        while len(supplier_id) != 4 or not supplier_id[0].isalpha() or not
supplier_id[1:].isdigit():
            print("Wrong supplier ID format, please try again")
            break

        name = input("Please enter the supplier's name: ")
        while not name.isalpha():
            print("Wrong name format. Please enter a name containing only letters and no
spaces.")
            name = input("Please enter the supplier's name: ")

        contact = input("Please enter the supplier's contact: ")
        while len(contact) != 10 or not contact.isdigit():
            print("Wrong contact format, please try again")
            contact = input("Please enter the supplier's contact: ")

        add_supplier(supplier_id, name, contact) #Add supplier with duplication check
        read_supplier() #Immediately update and print supplier details

        repeat = input("Do you want to continue adding supplier? [Yes/No]: ").lower()
        if repeat != "yes":
            print("-----")
            view_supplier()
            break

```

3. Place an Order

```

def place_order():
    try:
        # Read products from file
        with open(PRODUCTS_FILE, "r") as file:
            products = file.readlines()

        if not products:
            print("\nError: No products available in the inventory.")
            return

        print("\nAvailable Products:")
        print(f"{'Product ID':<12} {'Name':<20} {'Description':<30} {'Price':<10} {'Stock':<10}")
        print("-" * 82)
        for line in products:
            product_id, name, description, quantity, price = line.strip().split(',')
            description = description[:24] + "..." if len(description) > 25 else description

        print(f"{'product_id':<12} {'name':<20} {'description':<30} {'price':<10} {'quantity':<10}")

```

```

    # Prompt for product ID and quantity
    product_id_input = input("\nEnter product ID to order: ").strip()
    try:
        quantity = int(input("Enter quantity to order: ").strip())
        if quantity <= 0:
            print("\nError: Quantity must be a positive number.")
            return
    except ValueError:
        print("\nError: Quantity must be a valid number.")
        return

    # Process the order
    product_found = False
    updated_products = []

    for line in products:
        product_id, name, description, quantity_str, price = line.strip().split(',')
        if product_id == product_id_input:
            product_found = True
            current_stock = int(quantity_str)

            if current_stock >= quantity:
                new_stock = current_stock - quantity

    updated_products.append(f'{product_id},{name},{description},{new_stock},{price}\n')

    # Record the order
    with open(ORDERS_FILE, "a") as orders_file:
        order_id = f'ORD{len(products) + 1:03d}'
        order_date = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
        orders_file.write(f'{order_id},{product_id},{quantity},{order_date}\n')

    print(f"\nOrder placed successfully! Order ID: {order_id}")
    else:
        print("\nError: Insufficient stock available!")
        updated_products.append(line)
    else:
        updated_products.append(line)

    if not product_found:
        print("\nError: Product ID not found.")

    # Write updated product data back to the file
    with open(PRODUCTS_FILE, "w") as file:
        file.writelines(updated_products)

except FileNotFoundError:
    print("\nError: Required file not found. Ensure 'products.txt' exists.")
except Exception as e:
    print(f"\nAn unexpected error occurred: {e}")

```

4. View Inventory

```
def view_inventory():
    PRODUCTS_FILE = "products.txt" # Define the file name

    with open(PRODUCTS_FILE, 'r') as file:
        products = file.readlines()

    if products:
        print("Current Inventory:")
        print(f'{"Product ID":<10} {"Name":<20} {"Description":<30} {"Stock":<10} {"Price":<10}')
        print("-" * 82)

        for line in products:
            product_data = line.strip().split(',')
            if len(product_data) == 5: # Ensure the line is properly formatted
                product_id, name, description, quantity, price = product_data
                # Truncate description if it's too long
                description = description[:24] + "..." if len(description) > 30 else description
                print(f'{"product_id":<10} {"name":<20} {"description":<30} {"quantity":<10} {"price":<10}')
            else:
                print("The inventory is empty.")
```

5. Generate Reports

```
def generate_reports():
    """
    Generates reports for the inventory management system.
    Reports include:
    1. Low stock items
    2. Product sales
    3. Supplier orders
    """
    try:
        # Generate low stock items report
        print("\nLow Stock Items:")
        try:
            with open(PRODUCTS_FILE, 'r') as file:
                products = file.readlines()

            low_stock_items = []
            for line in products:
                product_data = line.strip().split(',')
                if len(product_data) == 5: # Ensure the line is properly formatted
                    product_id, name, description, quantity, price = product_data
                    if int(quantity) < 5: # Threshold for low stock
```



```

        low_stock_items.append((product_id, name, quantity))

    if low_stock_items:
        for item in low_stock_items:
            print(f"ID: {item[0]}, Name: {item[1]}, Quantity: {item[2]}")
        else:
            print("No low stock items found.")
    except FileNotFoundError:
        print("Error: 'products.txt' file not found.")

# Generate supplier orders report
print("\nSupplier Orders:")
try:
    with open('suppliers.txt', 'r') as file:
        suppliers = file.readlines()

    if suppliers:
        # Skip the first line (header)
        for line in suppliers[1:]: # Skip the header
            line = line.strip()
            # Split by whitespace (spaces or tabs)
            supplier_data = line.split()

            # Check if the line has exactly 3 parts
            if len(supplier_data) == 3:
                supplier_id, name, contact = supplier_data
                print(f"Supplier ID: {supplier_id}, Name: {name}, Contact: {contact}")
            else:
                print(f"Malformed line: {line}")
        else:
            print("No supplier data found.")
    except FileNotFoundError:
        print("Error: 'suppliers.txt' file not found.")
except Exception as e:
    print(f"An error occurred: {e}")

```

6. Exit Function

#1.7 Exit and Main Program

```

def main_program():
    continue_program = "

    while continue_program != 'n':
        print("\nInventory Management System")
        print('[1] Add a New Product')
        print('[2] Update Product Details')
        print('[3] Add a New Supplier')
        print('[4] Place an Order')
        print('[5] View Inventory')

```

```
print('[6] Generate Reports')
print('[7] Exit the program')

try:
    selection = int(input('Enter your selection: '))
    match selection:
        case 1:
            add_product()
        case 2:
            update_product_details()
        case 3:
            add_supplier_interactive()
        case 4:
            place_order()
        case 5:
            view_inventory()
        case 6:
            generate_reports()
        case 7:
            break
        case _:
            print('Invalid selection. Kindly, choose a number between 1 and 7, thank you.')
except ValueError:
    print('Error: Please enter a valid integer for menu selection.')

continue_program = input('Do you want to continue the program [y/n]: ').lower()
while continue_program not in ['y', 'n']:
    print('Invalid input. Please key in "y" for yes or "n" for no.')
    continue_program = input('Do you want to continue the program [y/n]: ').lower()

print('Successfully using Inventory Management System, thank you. Goodbye!')

# Run the program
main_program()
```